

Natural Language Processing Homework 2 Report

Setup:

I added one helper class, SharedBlockedCells, which masks blocked cells on every map with '.' (grass). After a goose passes through a door, the LLM returns that door's coordinates so it is masked and the geese never pay attention to the passed door again.

Planner storage memory: current phase, planner system prompt, goal position, last reasoning, completed_buttons (buttons that opened a door), tested_for_current_door buttons, door_to_check (the door we try to open now), the shared SharedBlockedCells instance, and passed_doors.

Each goose storage memory: positions history, goose system prompt, the symbol it is standing on, the shared SharedBlockedCells instance, 5x5 closest cells map (all cells within distance 2).

Methodology:

On every step of the Planner's step() function I first call each goose with the current phase and its target position. A goose walks to the target and returns its position, the symbol it stands on, the geese it sees, its last 3 positions, a closest cells map and the full numbered map. Then I call goose_1 again with an observe instruction, so it reports fresh information after goose_2 has moved.

To build the Planner user prompt I take both geese reports and add the current phase, the known goal position (found by the LLM), completed_buttons, tested_for_current_door, door_to_check, and a phase-specific instruction telling the model what to do and when to switch to which phase. The model answers with one JSON object giving the next phase, both geese target positions, its reasoning and some additional information if needed (goal positions, the button tried in the current turn, the door to check, and the cell to block).

Example of Planner user prompt and Planner answer (I am printing them in terminal during execution):

raw planner user prompt:

```
Current phase: goose2_move_to_button_check
Known goals position: (1, 10)
Buttons that WORKED (completed_buttons, never reuse): [(1, 4)]
Buttons already tried for the current door (tested_for_current_door, skip these): [(1, 6)]
Door to check: (4, 3)
```

Goose 1 info:

```
My current position: (4, 4), standing on .
Visible geese positions: {'goose_2': (1, 2), 'goose_1': (4, 4)}
My last 3 positions: [(4, 4), (4, 4), (4, 4)]
Closest cells map:
```

col: 23456

r 2:

r 3: #####

r 4: ./X\$?

r 5: #####?

Visible map:

col: 01234567890

r 0: ?#####?

r 1: #.Y.@.@..*#

r 2: #.....#

r 3: ?###.#####?

r 4: #../X\$?.???

r 5: ?#####?????

Goose 1 previous target position: (4, 4)

Goose 2 info:

```
My current position: (1, 2), standing on @
Geese (which I see) positions: {'goose_2': (1, 2), 'goose_1': (4, 4)}
My last 3 positions: [(1, 6), (1, 2), (1, 2)]
```

Closest cells map:

```
col: 01234
r 0: ?####
r 1: #.Y.@
r 2: #....
r 3: ?##.#
```

Visible map:

```
col: 01234567890
r 0: ?#####?
r 1: #.Y.@.@..*#
r 2: #.....#
r 3: ?##.#####?
r 4: #../X$?.???
r 5: ?####?~~~~
```

Goose 2 previous target position: (1, 2)

PHASE-SPECIFIC INSTRUCTIONS FOR "goose2_move_to_button_check":

- KEEP target_position_goose_2.
- CHECK 'door_to_check' position:

LOOK at goose_1's CLOSEST CELLS MAP!! Find the door ('\$` or `/') closest to goose_1 ON CLOSEST CELLS MAP, NEVER pick a door that is in the "Passed doors" list

CHECK if it in "door_to_check", if not change 'door_to_check' to this door ('\$` or `/') position. Set target_position_goose_1 to a cell adjacent to that door

(up/down/left/right) and closest to goose_1 (it might be goose_1 position, if goose_1 already standing in the next cell to '\$` or `/' cell). IF GOOSE_1 STANDING ON '/' or '\$`, CHANGE GOOSE_1 POSITION ACCORDING TO PREVIOUS RULES!!!

- If goose_2 is ON the button -> next_phase = "goose2_hold",
else next_phase = "goose2_move_to_button_check" and keep "target_position_goose_2" and keep "target_position_goose_1".
- Do NOT include tried_button_position or blocked_cell here.

NEVER PICK THIS COORDINATES as door_to_check: [(4, 7)] !!!!!!!!!!!!!!!!

Calling LLM, attempt 1

system size: 1515

Planner raw: ``json

```
{
  "reasoning": "Goose 1 is at (4, 4) and sees a door at (4, 3) in its closest cells map. This matches the 'door_to_check'. Goose 1 is already adjacent to (4, 3), so target_position_goose_1 remains (4, 4). Goose 2 is currently standing on the button at (1, 2), which is a potential trigger for the door. Therefore, we move to the hold phase to verify if the door (4, 3) opens.",
  "next_phase": "goose2_hold",
  "goal_position": "(1, 10)",
  "target_position_goose_1": "(4, 4)",
  "target_position_goose_2": "(1, 2)",
  "door_to_check": "(4, 3)"
}
```

Then I record the needed information from the JSON into the Planner storage and repeat.

Here's a quick overview of how your phases work (some details and verifications are omitted):

start – planner finds the target *, buttons @, and doors \$; decides which goose is closest to the buttons, one goes to press (gooseN_move_to_button), the other stands at the nearest door (door_to_check).

gooseN_move_to_button - if there are no buttons near gooseN -> start phase, otherwise the gooseN moves to the button, the other goes to the closest to him cell in front of the door, then gooseN_move_to_button_check phase.

gooseN_move_to_button_check - double-check target positions - one should be on the button, other on cell before the door, then gooseN_hold phase.

gooseN_hold - look at *door_to_check*: if \$ (closed) -> the button is wrong, mark it in tested. If there are buttons near *gooseN*-> *gooseN_move_to_button*, else -> *goose(another)N_move_to_buttons*. if / (open) -> the button works, the another goose goes to the far side of the door -> *gooseN_hold_hold*.

gooseN_hold_hold - mark the passed door as . , record the triggered button in *completed_buttons*. Next we check only the 4 adjacent cells of each goose: is there another \$ nearby? -> again in *move_to_button* phase for the desired goose; no -> *both_to_goal*.

both_to_goal - if the goose doesn't have an adjacent \$ -> both go to the goal; If there's an adjacent \$ -> return to appropriate *move_to_button* phase.

honk - both honk.

Protection: unknown phase → reset to start; *door_to_check* is never taken from *passed_doors*.

Agent Architecture:

The Planner has no tools and only reasons in JSON; each goose has two tools, move (up/down/left/right) and honk, and runs its own short tool-calling agentic loop(with max 10 iterations) trying to reach the target.

Inside a goose, *_tool_move* first reads the cell symbol it is about to enter on the masked map, performs the move, and only then updates *_standing_on* with that symbol (if the move was successful). The goose keeps *positions_history* and reports its last three positions, which lets the Planner detect when a goose is stuck repeating the same cell or went through the door.

To save model calls the goose skips the LLM entirely in passive situations: in the start phase, in its own hold phase, and in the extra observe call. In those cases it does not move and only returns its current report.

How Planner and Geese Cooperate:

The Planner is the only decision maker; the geese are executors. The geese never talk to each other directly, they cooperate only through the Planner: the Planner collects the information each goose reports, decide on phase and sends each goose where to go.

How I handled partial observability:

The Planner cannot see the grid itself, so the geese are its eyes. It receives the maps from both geese in the same prompt and can therefore tell which goose is closer to an untested button or door, and whether any are left.

Achieved reproducibility:

The LLM often answers the same question with the same data incorrectly. So I tried to anticipate the mistakes that might occur, collected them during experiments, and made the LLM double-check its decisions in the following phases. If something is wrong, the LLM can return to one of the previous phases and repeat until every condition is met. Furthermore, introducing agentic loop for geese boosted stability and reproducibility. This approach gave reproducible behavior in the sense that the geese are far more likely to finish a level than not (I did not calculate the exact probability, as it would take too long). In my experiments the geese successfully completed all 5 levels in both modes (easy and hard) (you can find evidence for highest levels in attached files, didn't manage to rerun levels 1 and 2 to get evidence, because I've exceeded daily limits in all my accounts).

Encountered Problems(beside described above):

The model miscounted coordinates and hallucinated where the '\$' and '@' were on the map, so I added the *_numbered_map* function, strict coordinate rules and examples with worked-out coordinates. The model also wanted to leave opened every door before going to the goal, so I made it look only at closed doors directly adjacent to a goose. Two-digit columns misaligned the map header and mislead the model, which I fixed with a modulo-10 label. In addition, Introducing agentic loops significantly increased requests and used tokens. I tried to deal with this problem by introducing if statements with no LLM request for some phases in geese class.

Insights:

The biggest improvements came from the *_numbered_map* function, from telling the model which symbol each goose is standing on, demanding to give reasoning in each step and from telling it which doors and buttons have already been tested. Splitting the work into phases with strict rules, and re-checking the same condition in more than one phase, also helped, as did warning the model that some information may be mistaken or misleading. Explicitly forbidding unpleasant behavior and using CAPS in the instructions helped a lot: sometimes the model understood a rule but then wrote something like "Yes, I know I should do it like this. However, ..." and invented its own rule, so making the key rules visually strong reduced this behaviour.